

Link to code: <https://github.com/luca-miniati/ds4400/blob/main/hw4/hw4.ipynb>

Solution 1

1. train error: 0.000290, train accuracy: 0.999710, train F1: 0.999633, train AUC: 1.000000
test error: 0.071242, test accuracy: 0.928758, test F1: 0.910675, test AUC: 0.928781
The tree basically memorizes the training set (train error ≈ 0) but test error is higher, so it's overfitting.
2. train error: 0.000290, train accuracy: 0.999710, train F1: 0.999633, train AUC: 1.000000
test error: 0.095569, test accuracy: 0.904431, test F1: 0.876404, test AUC: 0.897670
Entropy had lower test error than Gini on this run.
3. Implemented in code. The depth with the lowest test error was 20 (min test error ≈ 0.0652).
I'd pick something in that range instead of an unlimited tree.

Solution 2

1. Random forest ($T = 10$) train error: 0.002899, train accuracy: 0.997101, train F1: 0.996329,
train AUC: 0.999953
test error: 0.052129, test accuracy: 0.947871, test F1: 0.932127, test AUC: 0.982669

Random forest ($T = 50$) train error: 0.000290, train accuracy: 0.999710, train F1: 0.999633,
train AUC: 0.999999
test error: 0.033884, test accuracy: 0.966116, test F1: 0.956032, test AUC: 0.988810

Random forest ($T = 100$) train error: 0.000290, train accuracy: 0.999710, train F1: 0.999633,
train AUC: 0.999999
test error: 0.033015, test accuracy: 0.966985, test F1: 0.957494, test AUC: 0.989260

Random forest ($T = 500$) train error: 0.000290, train accuracy: 0.999710, train F1: 0.999633,
train AUC: 1.000000
test error: 0.034752, test accuracy: 0.965248, test F1: 0.954955, test AUC: 0.988773

As T gets bigger the test metrics improve at first and then mostly flatten out.

2. Random forest test error is lower than the single decision tree from problem 1 and test AUC is higher.
3. Implemented in code.

Solution 3

1. AdaBoost with stumps ($T = 10$) train error: 0.091304, train accuracy: 0.908696, train F1: 0.882419, train AUC: 0.964278
test error: 0.093831, test accuracy: 0.906169, test F1: 0.878104, test AUC: 0.962696

AdaBoost ($T = 50$) train error: 0.066087, train accuracy: 0.933913, train F1: 0.913636, train AUC: 0.982135
test error: 0.066898, test accuracy: 0.933102, test F1: 0.910361, test AUC: 0.979293

AdaBoost ($T = 100$) train error: 0.057681, train accuracy: 0.942319, train F1: 0.925663, train AUC: 0.984699
test error: 0.056473, test accuracy: 0.943527, test F1: 0.925884, test AUC: 0.981496

AdaBoost ($T = 500$) train error: 0.044638, train accuracy: 0.955362, train F1: 0.942793, train AUC: 0.990647
test error: 0.051260, test accuracy: 0.948740, test F1: 0.933333, test AUC: 0.985234

2. For the same T , random forest usually has higher test AUC than AdaBoost here. Both get better as T increases.
3. Implemented in code.

Solution 4

1. Implemented in code.
2. Implemented in code.
3. Custom NB test accuracy: 0.9562, precision (poisonous): 0.9923, recall: 0.9162, F1: 0.9527.
4. sklearn CategoricalNB with `alpha=1` gives the same accuracy, precision, recall, and F1.